GrowthOS Ads Module: Multi-Phase Architecture and Implementation Plan

Document Metadata

- **Author**: AI Engineering & Systems Architecture
- **Target Module**: GrowthOS Module 3 — Ads Layer Manager (Google Ads Search Connector)
- **Consumers**: Module 4 (AI Growth Brain), Module 2 (Behavior Intelligence SDK), Merchant Cockpit
- **Status**: Approved Implementation Plan (Draft v1)
- **Date**: October 2026

---

1. Google Cloud Console & Google Ads Console Setup Guide

You have a Google Cloud Console project with the Google Ads API enabled and an **Explorer (Test)** access Developer Token. Here is the step-by-step setup required to make authenticated API requests.

1.1 Understanding Explorer / Test Access

- A developer token with **Test / Explorer access** is restricted by Google: it can **only execute API calls against Google Ads Test Accounts**.
- It **cannot** interact with production Google Ads accounts until upgraded to Basic or Standard access.
- However, test accounts expose the exact same GAQL queries, search stream, and mutate operations as production accounts, making them ideal for development, testing, and continuous integration.

1.2 Step 1: Google Cloud Console OAuth 2.0 Credentials

1. Navigate to the **Google Cloud Console** (`console.cloud.google.com`) and select your project where Google Ads API is enabled.
2. Go to **APIs & Services > OAuth consent screen**:
    - **User Type**: Select **External** (unless you are using Google Workspace internally).
    - **App Information**: Enter `GrowthOS Ads Connector` and your support email.
    - **Scopes**: Click **Add or Remove Scopes**, filter for `Google Ads API` , and add:
        - `https://www.googleapis.com/auth/adwords`
    - **Test Users**: Under the Test Users section, add the email address of the Google account you will use to manage the Google Ads test account. Click **Save and Continue**.
3. Go to **APIs & Services > Credentials**:
    - Click **Create Credentials > OAuth client ID**.
    - **Application type**: Select **Desktop app** (recommended for local scripts/backend worker authentication) or **Web application**.
    - If selecting **Web application**, configure Authorized Redirect URIs:
        - `http://localhost:8000/api/v1/ads/oauth/callback`
        - `https://developers.google.com/oauthplayground` (useful for quick token generation).
    - Click **Create**.
    - Copy and save the **Client ID** and **Client Secret**.

1.3 Step 2: Create Google Ads Test Manager & Client Accounts

A standard live Google Ads account will return `DEVELOPER_TOKEN_NOT_APPROVED` when accessed with an Explorer token. You must create a Google Ads Test Account:

1. Go to the Google Ads Test Manager Account Creation Page (make sure you use the test creation link: `https://ads.google.com/home/tools/manager-accounts/` with test mode or `https://developers.google.com/google-ads/api/docs/first-call/test-accounts`).

2. Sign in with the Google Account added as a Test User in Step 1.2.
3. Create the test manager account (e.g., `GrowthOS Test MCC`). The test account UI will show a red banner at the top: "**This is a test account**".
4. Inside this Test Manager Account, create a **Test Client Account**:
    - Click the **Accounts** tab > **+** (New Account) > **Create new account**.
    - Name: `Dominos India Demo Store (Test)`.
    - Currency: `INR`.
    - Time Zone: `(GMT+05:30) India Standard Time`.
5. Note the two numeric IDs (10 digits without hyphens):
    - **Login Customer ID** (`login_customer_id`): The 10-digit ID of the Test Manager (MCC) Account.
    - **Client Customer ID** (`client_customer_id`): The 10-digit ID of the Test Client Account.

1.4 Step 3: Generate the OAuth 2.0 Refresh Token

To allow background workers and sync services to query Google Ads without manual user re-login, obtain a permanent refresh token.

Option A: Using the Google Ads Python Script (Recommended)

1. In your local virtual environment, install the Google Ads client:

```
pip install google-ads
```

2. Download or run Google's credential generator helper:

```
python3 -m google.ads.googleads.generate_user_credentials --client_id=YOUR_CLIENT_ID --client_secret=YOUR_CLIENT_SECRET
```

3. A browser window will open requesting consent. Log in with your Test User account and grant permissions.
4. The terminal will output your `refresh_token`.

Option B: Using Google OAuth 2.0 Playground

1. Go to `https://developers.google.com/oauthplayground`.
2. Click the gear icon (top right): check **Use your own OAuth credentials** and enter your `client_id` and `client_secret`.
3. In Step 1 (Select & authorize APIs), paste `https://www.googleapis.com/auth/adwords` and click **Authorize APIs**.
4. In Step 2 (Exchange authorization code for tokens), click **Exchange authorization code for tokens**.
5. Copy the value of the `refresh_token`.

1.5 Step 4: Environment Configuration

Create your configuration file in `research/Ads-Module/.env`:

```
# Google Ads API Credentials (Explorer / Test Level)
GOOGLE_ADS_DEVELOPER_TOKEN="your_explorer_developer_token"
GOOGLE_ADS_CLIENT_ID="your_client_id.apps.googleusercontent.com"
GOOGLE_ADS_CLIENT_SECRET="your_client_secret"
GOOGLE_ADS_REFRESH_TOKEN="your_refresh_token"
GOOGLE_ADS_LOGIN_CUSTOMER_ID="1234567890"    # MCC Manager ID (no dashes)
GOOGLE_ADS_CLIENT_CUSTOMER_ID="0987654321"   # Test Client Account ID (no dashes)
GOOGLE_ADS_USE_PROTO_PLUS=True

# Ads Module Environment
ADS_MODULE_PORT=8001
ADS_MODULE_DB_URL="sqlite:///./ads_module.db"
```

```
AI_MODULE_WEBHOOK_URL="http://localhost:8000/v1/telemetry/events"
WORKSPACE_ID="tenant_dominos_qsr"
```

---

2. Multi-Phase Implementation Plan Overview

The Ads Module implementation is partitioned into six clear, sequential phases:

```
[ Phase 0: Credentials, Environment & Mock Fixture Harness ]
                          |
                          v
[ Phase 1: Barebone Pipeline (Minimal MVP) ]
  - Connector (GAQL search stream)
  - 12-Field Normalization Layer
  - Storage Layer (ads_campaigns_daily)
  - Sync Worker (Idempotent 2-hour pull)
  - REST API Endpoints & ads.metrics.updated publisher
                          |
                          v
[ Phase 2: AI Module Contract & Metrics Integration (Read-Path) ]
  - Blended CAC & ROAS in MetricsEngine
  - Anomaly detectors (spend spike, CPA drift, pacing breaks)
  - DigestEngine narration without mathematical hallucination
                          |
                          v
[ Phase 3: Campaign Planning, Validation & State Machine Deployment ]
  - Natural-language to campaign specification
  - Pre-deployment validation engine
  - State machine (DRAFT -> AWAITING_APPROVAL -> DEPLOYING -> ACTIVE)
  - validate_only dry runs on Google Ads API
                          |
                          v
[ Phase 4: Closed-Loop Attribution Bridge ]
  - Ingest gclid & UTM parameters from Behavior SDK
  - Multi-touch & last-touch session-to-order attribution
  - Enhanced / Offline conversion upload to Google Ads
                          |
                          v
[ Phase 5: Autonomous Optimization & Policy-Gated Write-Back ]
  - Budget shifts & underperforming ad pausing
  - PolicyEngine strict limits (15% daily shift cap, integer paise)
  - ActionOrchestrator holdout tracking & audit logging
```

---

3. Phase-by-Phase Component Breakdown

Phase 0: Setup, Credentials & Offline Playback Harness

- **Objective**: Establish the development workspace, test account credentials, and build an offline fixture recorder/player so the entire test suite runs deterministically without network flakiness.

- **Components to Build**:
  1. `research/Ads-Module/config.py` : Environment configuration reader using Pydantic Settings.
  2. `research/Ads-Module/connectors/fixtures/` : Recorded Google Ads API JSON/Protobuf responses for campaigns, ad groups, and keyword performance rows.
  3. `research/Ads-Module/connectors/playback_client.py` : Offline fallback client that replicates Google Ads GAQL responses for testing.
- **Deliverable**: Passing smoke test connecting to live Google Ads API (when credentials are provided) with transparent fallback to recorded fixtures.

Phase 1: Barebone Pipeline (Minimal MVP)

- **Objective**: Build the minimum viable data flow that pulls data from Google Ads, normalizes it into GrowthOS schema, stores it, and exposes it via REST endpoints and events.
- **Components to Build**:

1. Account Connection Service & Models

- **Location**: [research/Ads-Module/domain/models.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/domain/models.py)
- **Entities**:
  - `AccountConnection` : Workspace ID, account ID, platform (`google_ads`), status (`ACTIVE` , `AUTHENTICATION_FAILED`), encrypted refresh token reference, currency (`INR`), timezone (`Asia/Kolkata`).
  - `CampaignRecord` : Campaign ID, account ID, name, status (`ENABLED` , `PAUSED`), channel type (`SEARCH`), daily budget in integer paise.
  - `AdsCampaignDaily` : The canonical 12-field normalized schema.

2. Google Ads Connector

- **Location**: [research/Ads-Module/connectors/google_ads_connector.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/connectors/google_ads_connector.py)
- **Responsibilities**:
  - Implements `AdsConnectorBase` protocol.
  - Executes GAQL (Google Ads Query Language) queries over `GoogleAdsService.SearchStream` .
  - Queries `campaign` , `metrics` , and `segments` entities for dates, spend, impressions, clicks, conversions, and status.
  - Handles rate limits with exponential backoff (ladder: 30s, 2m, 10m).

3. Normalization Layer

- **Location**: [research/Ads-Module/normalization/normalizer.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/normalization/normalizer.py)
- **Conversion Rules**:
  - **Spend**: Converts Google's `metrics.cost_micros` (millionths of account currency) to **integer paise** (1 INR = 100 paise) and decimal currency (`cost_micros / 1,000,000`).
  - **Status**: Maps Google statuses (`ENABLED` , `PAUSED` , `REMOVED`) to GrowthOS states (`ACTIVE` , `PAUSED` , `ARCHIVED`).
  - **Timezones**: Normalizes date boundaries relative to the account's configured timezone (`Asia/Kolkata` pinned to IST midnight).
  - **Derived Ratios**:

$$\text{CTR} = \frac{\text{clicks}}{\text{impressions}}$$

$$\text{CPC} = \frac{\text{spend}}{\text{clicks}}$$

$$\text{CPA} = \frac{\text{spend}}{\text{conversions}}$$

$$\text{ROAS} = \frac{\text{conversion_value}}{\text{spend}}$$

4. Storage Layer

- **Location**: [research/Ads-Module/storage/ads_storage.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/storage/ads_storage.py)

- **Schema (`ads_campaigns_daily`)**:
  - `workspace_id` (TEXT)
  - `platform` (TEXT, e.g. "google_ads")
  - `account_id` (TEXT)
  - `campaign_id` (TEXT)
  - `campaign_name` (TEXT)
  - `date` (TEXT, ISO-8601 YYYY-MM-DD)
  - `spend_paise` (INTEGER)
  - `spend_inr` (REAL)
  - `impressions` (INTEGER)
  - `clicks` (INTEGER)
  - `conversions` (REAL)
  - `conversion_value_paise` (INTEGER)
  - `ctr` (REAL)
  - `cpc_paise` (INTEGER)
  - `cpa_paise` (INTEGER)
  - `roas` (REAL)
  - `synced_at` (TEXT)
- **Capabilities**: Idempotent upserts on composite primary key `(workspace_id, platform, account_id, campaign_id, date)` .

5. Sync Worker Service

- **Location**: [research/Ads-Module/services/sync_service.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/services/sync_service.py)
- **Responsibilities**:
  - Idempotent scheduled execution (runs every 2 hours).
  - Pulls previous 3 days of metrics to absorb Google's delayed conversion adjustments.
  - Emits `ads.metrics.updated` event upon successful completion.

6. REST API Endpoints

- **Location**: [research/Ads-Module/api/routes.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/api/routes.py)
- **Endpoints**:
  - `POST /api/v1/ads/accounts/connect` : Registers customer ID and credentials.
  - `GET /api/v1/ads/accounts` : Lists active account connections.
  - `POST /api/v1/ads/accounts/{account_id}/sync` : Triggers on-demand sync.
  - `GET /api/v1/ads/campaigns/{campaign_id}/metrics` : Retrieves daily performance rows for a specific campaign.
  - `GET /api/v1/ads/campaigns/metrics/bulk` : Bulk fetch for a workspace filtered by date window.

7. Event Publisher

- **Location**: [research/Ads-Module/events/publisher.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/events/publisher.py)
- **Event**: Emits `ads.metrics.updated` payload containing `workspace_id` , `account_id` , `date_start` , `date_end` , and `record_count` .

Phase 2: AI Module Read-Path Integration & Contract Tests

- **Objective**: Connect the normalized performance data into the AI Module ([research/ai-module](file:///home/mihir/Codes/internship/GrowthOS/research/ai-module)) so that `MetricsEngine` and `DigestEngine` compute ad analytics and narrate findings deterministically.
- **Components to Build & Wire**:
  1. `research/ai-module/ports/ads_port.py` : Formal interface defining `fetch_campaign_metrics()` .
  2. `research/ai-module/adapters/http_ads_adapter.py` : HTTP client implementing `ads_port.py` , querying `GET /api/v1/ads/campaigns/metrics/bulk` .
  3. `research/ai-module/engines/metrics_engine.py` Extensions:

- Ad metrics: Blended Customer Acquisition Cost ($\text{CAC} = \frac{\text{Ad Spend}}{\text{New Customers}}$), Blended ROAS ($\frac{\text{Attributed Revenue}}{\text{Ad Spend}}$), Budget Pacing ($\frac{\text{Spend to Date}}{\text{Monthly Budget}}$).

4. Anomaly Detectors for Ads:
   - Saturday-evening CPA spikes.
   - Sudden drop in impression share (>30% week-over-week drop).
   - Budget exhaustion before 14:00 IST.
5. `research/ai-module/engines/digest_engine.py` :
   - Narrates ad performance findings.
   - Enforces that the LLM is fed pre-calculated numbers, never generating metrics independently.
6. Contract Tests ([research/ai-module/tests/contracts/test_ads_contract.py](file:///home/mihir/Codes/internship/GrowthOS/research/ai-module/tests)):
   - Validates that API schemas match the expected 12 fields, integer paise, and date formatting.

Phase 3: Campaign Planning, Validation & State Machine Deployment

- **Objective**: Transform natural language marketing goals into structured campaign proposals, validate them against merchant and Google constraints, and deploy them through a state-machine workflow.
- **Components to Build**:

1. Campaign Planning Engine

- **Location**: [research/Ads-Module/engines/planning_engine.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/engines/planning_engine.py)
- **Input**: Natural language query (e.g. *"Increase weekend pizza orders from college students in Mumbai, target CPA under INR 250"*).
- **Output**: Structured `CampaignPlan` :
  - Objective (`SALES` , `LEADS` , `WEBSITE_TRAFFIC`)
  - Target Geography (`Mumbai` , postal codes, or radius)
  - Budget (`daily_budget_paise: 500000` = INR 5,000)
  - Bidding Strategy (`TARGET_CPA` , target: 25000 paise)
  - Keyword Clusters (High intent: `order pizza online` , `pizza delivery near me` ; Competitor; Negative keywords: `free pizza recipes` , `jobs`)
  - Ad Copy (Headlines ≤ 30 chars, Descriptions ≤ 90 chars, Destination URL)

2. Validation Engine

- **Location**: [research/Ads-Module/engines/validation_engine.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/engines/validation_engine.py)
- **Guardrails**:
  - Verifies budget meets Google Ads minimums and merchant daily caps.
  - Checks destination URL validity and HTTPS status.
  - Enforces headline character limits (≤ 30) and description limits (≤ 90).
  - Prevents duplicate active campaign names.
  - Flags missing negative keywords.

3. Approval and Deployment State Machine

- **Location**: [research/Ads-Module/services/deployment_service.py](file:///home/mihir/Codes/internship/GrowthOS/research/Ads-Module/services/deployment_service.py)
- **State Machine**:

$$\text{DRAFT} \longrightarrow \text{VALIDATING} \longrightarrow \text{AWAITING_APPROVAL} \longrightarrow \text{APPROVED} \longrightarrow \text{DEPLOYING} \longrightarrow \text{ACTIVE}$$

  - Failure states: `VALIDATION_FAILED` , `DEPLOYMENT_FAILED` , `PLATFORM_REJECTED` .
- **Execution**:
  - Uses `CampaignOperation` mutates.
  - Performs dry runs using Google's `validate_only = True` flag before actual execution.

- In non-production or test mode, logs the dry-run response without live spend risk.

Phase 4: Closed-Loop Attribution Bridge

- **Objective**: Link Google ad clicks to on-site checkouts and purchases captured by the Behavior SDK, enabling true CAC/ROAS calculations and offline conversion upload.
- **Components to Build**:
  1. `research/Ads-Module/attribution/click_tracker.py`:
     - Maps `gclid` (Google Click ID) and UTM tags (`utm_source`, `utm_campaign`, `utm_medium`, `utm_term`) from incoming SDK events to campaign records.
  2. `research/Ads-Module/attribution/conversion_matcher.py`:
     - Matches order completion events (`order_completed`) back to the initial `gclid` within the configured lookback window (default 30 days, or 24h short window).
  3. `research/Ads-Module/connectors/offline_conversion_uploader.py`:
     - Prepares `ClickConversion` payloads for Google Ads `ConversionUploadService`.
     - Uploads hashed order IDs, transaction values in integer paise, and conversion timestamps to Google to train automated smart bidding.

Phase 5: Autonomous Optimization & Policy-Gated Write-Back

- **Objective**: Allow GrowthOS to recommend and safely execute optimizations (reallocating budget between campaigns, pausing poor performers, adding negative keywords).
- **Components to Build**:
  1. `research/Ads-Module/engines/optimization_engine.py`:
     - Detects high-CPA bleeding campaigns ($\text{CPA} > 1.5 \times \text{Target CPA}$ with statistically significant sample).
     - Proposes budget reallocation from underperforming campaigns to winning campaigns.
     - Identifies search term waste and generates negative keyword additions.
  2. Policy Gate & Action Orchestration Integration:
     - Routes all proposals through [PolicyEngine](file:///home/mihir/Codes/internship/GrowthOS/research/ai-module/engines/policy_engine.py).
     - Enforces the invariant: **Maximum 15% budget shift per 24 hours**.
     - Verifies action against the merchant's operating mode (`MANUAL`, `APPROVAL`, or `AUTONOMOUS`).
  3. Rollback & Audit Trail:
     - Records every budget change in an immutable audit ledger with before and after states.
     - Provides single-click rollback for reversible changes.

---

4. Component Mapping & File Tree

The final structure of the Ads Module:

```
research/Ads-Module/
├── docs/
│   ├── Core_Idea.md                          # Core concepts and PRD extraction
│   └── Multi_Phase_Implementation_Plan.md    # This comprehensive execution guide
├── config.py                                 # Pydantic Settings & environment loader
├── domain/
│   ├── models.py                             # Pydantic schemas (12-field daily metrics, accounts)
│   ├── events.py                             # ads.metrics.updated, ads.campaign.* events
│   └── plans.py                              # CampaignPlan, KeywordGroup, AdCopy schemas
├── connectors/
│   ├── base.py                               # Abstract AdsConnector interface
```

```
│   ├── google_ads_client.py                  # Live GAQL client (Google Ads API v17/v18)
│   ├── playback_client.py                    # Recorded fixture playback client
│   └── fixtures/                             # Recorded JSON/Protobuf responses
│       ├── campaign_search_stream.json
│       └── ad_group_search_stream.json
├── normalization/
│   └── normalizer.py                         # Micros to paise, timezone normalization, ratios
├── storage/
│   ├── schema.sql                            # DDL for ads_campaigns_daily and audit tables
│   └── ads_storage.py                        # SQLite / Postgres repository
├── services/
│   ├── sync_service.py                       # Periodic report pull & normalization worker
│   └── deployment_service.py                 # State machine for campaign deployment
├── engines/
│   ├── planning_engine.py                    # Natural language goal parsing
│   ├── validation_engine.py                  # Pre-deployment validation checks
│   └── optimization_engine.py                # Budget shift & negative keyword rules
├── attribution/
│   ├── click_tracker.py                      # gclid & UTM session attribution
│   └── conversion_uploader.py                # Google Ads ConversionUploadService client
├── api/
│   └── routes.py                             # FastAPI REST endpoints (/api/v1/ads/...)
├── tests/
│   ├── test_connector_playback.py           # Connector & GAQL query parsing tests
│   ├── test_normalization.py                 # Micros, paise, timezone & ratio tests
│   ├── test_sync_service.py                  # Sync worker & upsert tests
│   ├── test_validation_engine.py             # Budget & copy constraint tests
│   └── test_api_routes.py                    # FastAPI route contract tests
├── requirements.txt                          # google-ads, fastapi, sqlalchemy, pydantic
└── .env.example                              # Sample credential template
```

---

5. Summary of Immediate Next Steps

1. **Step 1 (User Action)**:
   - Follow the steps in Section 1 to create the Test Manager Account, Test Client Account, and generate your `refresh_token`.
   - Add the credentials to `research/Ads-Module/.env`.
2. **Step 2 (Implementation Execution)**:
   - Scaffold `research/Ads-Module/` with `requirements.txt` and domain models.
   - Build the **Barebone Pipeline** (Phase 1): Connector with fixture playback fallback, Normalization layer, Storage repository, Sync worker, and FastAPI endpoints.
   - Write and verify unit and contract tests.
3. **Step 3 (AI Integration)**:
   - Wire the AI Module's `MetricsEngine` and `DigestEngine` to consume the normalized `/api/v1/ads/campaigns/metrics/bulk` endpoint.